Week 10: Firewall Rules & Security

Date: May 11, 2026
Study Time: 5-8 hours total

🎯 GOAL

Implement custom firewall rules enforcing tier hierarchy

📚 STUDY TASKS (1-2 hours)

Reading

- [ ] **"How Linux Works" by Brian Ward**
  - Chapter 9: Firewalls section
  - Focus on: packet filtering, iptables basics

Video Learning

- [ ] **LearnLinuxTV** - Search: "iptables Tutorial"
  - https://youtube.com/@LearnLinuxTV
- [ ] **Professor Messer** - Search: "Linux+ Firewall"
  - https://youtube.com/@professormesser

Concepts to Understand

- [ ] What is a firewall?
- [ ] Stateful vs stateless firewalls
- [ ] iptables chains: INPUT, OUTPUT, FORWARD
- [ ] Default policies: ACCEPT vs DROP
- [ ] Rule matching order (top to bottom)
- [ ] UFW vs iptables vs nftables vs firewalld
- [ ] Port blocking vs IP blocking
- [ ] Connection tracking (ESTABLISHED, RELATED)

🔧 LAB WORK (3-4 hours)

Understanding Your Current Firewall

- [ ] **Check if UFW is active (Ubuntu/Debian)**

```bash
sudo ufw status
# If active, you'll see rules
# If inactive, firewall is disabled
```

- [ ] **Check if firewalld is active (Fedora/RHEL)**

```bash
sudo systemctl status firewalld
sudo firewall-cmd --state
```

- [ ] **View raw iptables rules**

```bash
sudo iptables -L -v -n

# Flags:
# -L = list rules
# -v = verbose (shows packet/byte counts)
# -n = numeric (don't resolve hostnames)

# You'll see three main chains:
# - INPUT (incoming traffic TO this machine)
# - FORWARD (traffic THROUGH this machine)
# - OUTPUT (outgoing traffic FROM this machine)
```

Understanding Chains and Rules

- [ ] **View INPUT chain only**

```bash
sudo iptables -L INPUT -v -n
```

- [ ] **View with line numbers**

```bash
sudo iptables -L INPUT -v -n --line-numbers
# Useful for inserting/deleting specific rules
```

- [ ] **Check default policies**

```bash
sudo iptables -L | grep policy
# Should show: Chain INPUT (policy ACCEPT)
# Or: Chain INPUT (policy DROP)
```

Basic iptables Rules

⚠️ WARNING: Always have physical access or a backup connection before modifying firewall rules! You can lock yourself out!

- [ ] **Allow SSH (CRITICAL - do this first!)**

```bash
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Breakdown:
# -A INPUT = append to INPUT chain
# -p tcp = protocol is TCP
# --dport 22 = destination port 22
# -j ACCEPT = jump to ACCEPT (allow packet)

# If you changed SSH to port 2222:
sudo iptables -A INPUT -p tcp --dport 2222 -j ACCEPT
```

- [ ] **Allow established connections (IMPORTANT)**

```bash
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# This allows responses to connections you initiated
# Without this, you can't browse websites, etc.
```

- [ ] **Allow loopback traffic**

```bash
sudo iptables -A INPUT -i lo -j ACCEPT

# -i lo = interface is loopback (127.0.0.1)
# Needed for local services to talk to each other
```

- [ ] **Allow HTTP and HTTPS (if running web server)**

```bash
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT   # HTTP
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT  # HTTPS
```

Implement Tier-Based Communication Restrictions

Your Goal: Devices cannot communicate UP the tier hierarchy

Example Tailscale IPs:

Tier 1: 100.64.0.1-3 (Laptop, Desktop, iPhone)
Tier 2: 100.64.0.10 (Fedora server)
Tier 3: 100.64.0.20 (Pi Zero bastion)
Tier 4: 100.64.0.30 (Pi 4 automation)

- [ ] **On Tier 2 (Fedora): Block incoming from Tier 3 & 4**

```bash
# SSH into Fedora server
ssh fedora

# Block Pi Zero (Tier 3) from connecting
sudo iptables -A INPUT -s 100.64.0.20 -j DROP

# Block Pi 4 (Tier 4) from connecting
sudo iptables -A INPUT -s 100.64.0.30 -j DROP

# Allow Tier 1 devices (Laptop, Desktop, iPhone)
sudo iptables -A INPUT -s 100.64.0.1 -j ACCEPT
sudo iptables -A INPUT -s 100.64.0.2 -j ACCEPT
sudo iptables -A INPUT -s 100.64.0.3 -j ACCEPT
```

- [ ] **On Tier 3 (Pi Zero): Block incoming from Tier 4**

```bash
ssh pi-zero

# Block Pi 4 from connecting
sudo iptables -A INPUT -s 100.64.0.30 -j DROP

# Allow Tier 1 & 2
sudo iptables -A INPUT -s 100.64.0.0/24 -j ACCEPT
```

- [ ] **Test tier restrictions**

```bash
# From Pi 4, try to SSH to Fedora (should fail)
ssh fedora
# Should timeout or be refused

# From Laptop, SSH to Fedora (should work)
ssh fedora
# Should connect fine
```

Block All Other Traffic (Secure Default)

- [ ] **Set default INPUT policy to DROP**

```bash
# ONLY do this after allowing SSH and established connections!
sudo iptables -P INPUT DROP

# Now everything not explicitly allowed is blocked
```

- [ ] **Test you can still connect**

```bash
# In a NEW terminal, try to SSH
ssh your-server
# Should still work!
```

Save Firewall Rules (They're Lost on Reboot!)

- [ ] **On Ubuntu/Debian**

```bash
# Install iptables-persistent
sudo apt install iptables-persistent

# Save current rules
sudo netfilter-persistent save

# OR manually:
sudo iptables-save > /etc/iptables/rules.v4

# Restore on boot (handled by iptables-persistent)
```

- [ ] **On Fedora/RHEL**

```bash
# Save rules
sudo iptables-save > /etc/sysconfig/iptables

# Or use firewalld instead (converts iptables to zones)
```

Alternative: Use UFW (Easier)

If iptables is too complex, UFW provides a simpler interface:

- [ ] **Enable UFW**

```bash
sudo ufw enable
```

- [ ] **Allow SSH first!**

```bash
sudo ufw allow 22/tcp
# Or if you changed port:
sudo ufw allow 2222/tcp
```

- [ ] **Allow from specific IP**

```bash
sudo ufw allow from 100.64.0.1
```

- [ ] **Deny from specific IP**

```bash
sudo ufw deny from 100.64.0.30
```

- [ ] **View rules**

```bash
sudo ufw status numbered
```

- [ ] **Delete a rule**

```bash
sudo ufw delete 3  # Delete rule number 3
```

Document Your Firewall Architecture

Create a document showing:

```
TIER 1 (Laptop, Desktop, iPhone)
↓ CAN connect to ↓
TIER 2 (Fedora Server)
  Firewall Rules:
  - ACCEPT from 100.64.0.1-3
  - DROP from 100.64.0.20, 100.64.0.30
  - ACCEPT SSH (port 22)
  - ACCEPT established connections

↓ CAN connect to ↓
TIER 3 (Pi Zero Bastion)
  Firewall Rules:
  - ACCEPT from Tier 1 & 2
  - DROP from 100.64.0.30
  - ACCEPT SSH (port 22)

↓ CAN connect to ↓
TIER 4 (Pi 4 Automation)
  Firewall Rules:
  - ACCEPT from Tier 1, 2, & 3
  - Acts as exit node
```

Lab Tasks Checklist

- [ ] View current iptables rules on all devices
- [ ] Allow SSH on all devices
- [ ] Allow established connections on all devices
- [ ] Implement tier-based blocking
- [ ] Test tier restrictions (Pi 4 can't reach Tier 2)
- [ ] Save firewall rules to survive reboot
- [ ] Reboot one device and verify rules persist
- [ ] Document firewall rules for each device

💻 FLASHCARD COMMANDS

Create flashcards for these commands:

1. `iptables`
2. `iptables -L`
3. `iptables -L -v -n`
4. `iptables -A` (append rule)
5. `iptables -D` (delete rule)
6. `iptables -P` (set policy)
7. `iptables-save`
8. `iptables-restore`
9. `nft`
10. `ufw`
11. `ufw allow`
12. `ufw deny`
13. `ufw status`
14. `firewalld`
15. `firewall-cmd`

Concepts for Flashcards:

- What are the three main chains in iptables?
- What's the difference between -A and -I in iptables?
- What does -j ACCEPT mean?
- What does -j DROP mean?
- What is the default policy?
- Why allow established connections?
- What is conntrack?

📖 EXAM OBJECTIVES

CompTIA Linux+ (XK0-005) Exam Objectives: https://partners.comptia.org/docs/default-source/resources/comptia-linux-xk0-005-exam-objectives-(3-0)

Relevant Sections:

- 3.3 Given a scenario, implement and configure Linux firewalls
- 3.4 Given a scenario, implement and execute security best practices
- 1.3 Given a scenario, configure and verify network connection parameters

---

📝 NOTES & REFLECTIONS

Firewall rules implemented on each device:

Tier 1 (Laptop):

Tier 2 (Fedora):

Tier 3 (Pi Zero):

Tier 4 (Pi 4):

Testing results:

- ☐ Tier 4 cannot reach Tier 2 ✓/✗
- ☐ Tier 3 cannot reach Tier 2 ✓/✗
- ☐ Tier 1 can reach all tiers ✓/✗

Issues encountered:

How did you recover from lockout? (if any)

Completed: ☐

🔗 QUICK REFERENCE

iptables Basic Syntax:

```bash
iptables [-t table] -[AID] chain rule-specification [options]
```

Common Options:

- `-A` = Append to end of chain
- `-I` = Insert at beginning
- `-D` = Delete rule
- `-p` = Protocol (tcp, udp, icmp)
- `-s` = Source IP
- `-d` = Destination IP
- `--sport` = Source port
- `--dport` = Destination port
- `-j` = Jump target (ACCEPT, DROP, REJECT)
- `-i` = Input interface
- `-o` = Output interface

Common Targets:

- `ACCEPT` = Allow packet
- `DROP` = Silently discard
- `REJECT` = Discard with error message
- `LOG` = Log packet and continue

Rule Order Matters!

```bash
# This blocks everything (rule 1 matches first):
sudo iptables -A INPUT -j DROP
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT  # Never reached!

# This works (SSH rule comes first):
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -j DROP
```

Emergency Recovery:

```bash
# If you lock yourself out, physical console access:

# Flush all rules (removes everything)
sudo iptables -F

# Set policies to ACCEPT
sudo iptables -P INPUT ACCEPT
sudo iptables -P OUTPUT ACCEPT
sudo iptables -P FORWARD ACCEPT

# Now you can SSH in and start over
```

Comparison: UFW vs iptables vs firewalld

Feature	UFW	iptables	firewalld
Ease of use	Easiest	Hard	Medium
Default on	Ubuntu	-	Fedora/RHEL
Persistent	Auto	Manual	Auto
Best for	Beginners	Advanced	RHEL users

Previous Week: Networking Fundamentals
Next Week: Centralized Logging (Part 1)